

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 - SEARCHING



NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Searching atau pencarian adalah proses menemukan data tertentu dari sekumpulan data yang tersedia. Algoritma pencarian dibagi menjadi dua jenis utama yaitu Sequential Search dan Binary Search. Data yang akan dicari perlu disimpan terlebih dahulu dalam struktur data array, yaitu struktur paling dasar yang menyimpan kumpulan data bertipe sama secara berurutan dalam memori. Dalam bahasa Go, array dapat berukuran tetap maupun dinamis menggunakan slice.

Sequential Search adalah algoritma pencarian paling sederhana yang memeriksa setiap elemen satu per satu dari indeks pertama hingga terakhir secara linear. Algoritma ini tidak mensyaratkan data terurut sehingga dapat diterapkan pada semua kondisi data. Jika ditemukan maka indeks posisinya dikembalikan, jika tidak maka dikembalikan nilai -1. Kompleksitas waktunya adalah $O(n)$ sehingga lebih cocok untuk data berukuran kecil.

Binary Search adalah algoritma pencarian yang lebih efisien namun mensyaratkan data harus sudah terurut. Algoritma ini membagi rentang pencarian menjadi dua bagian di setiap langkah menggunakan penanda batas kiri, kanan, dan tengah, lalu membandingkan nilai tengah dengan nilai yang dicari untuk menentukan arah pencarian berikutnya. Kompleksitas waktunya adalah $O(\log n)$ sehingga sangat efisien untuk data berukuran besar.

Struct dalam bahasa Go adalah tipe data komposit yang menggabungkan beberapa field berbeda tipe dalam satu kesatuan, seperti nama, NIM, dan IPK dalam satu variabel mahasiswa. Penggunaan struct dalam pencarian memungkinkan program mengakses seluruh atribut entitas yang ditemukan langsung melalui indeks hasil pencarian. Kombinasi struct, array, dan kedua algoritma pencarian mencerminkan konsep pemrograman modular di mana setiap fungsi memiliki tanggung jawab tunggal yang jelas.

B. Guided

1. [sequentialSearch.go]

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
}
```

```

        return found
    }

func main() {
    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {
        fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari : ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)
    } else if idxCari == -1 {
        fmt.Printf("Data %s tidak ditemukan!", binatangCari)
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\guided1.go"
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : kelinci
Masukkan data binatang indeks ke-3 : ayam
Masukkan data binatang indeks ke-4 : burung

Masukkan nama binatang yang mau dicari : kelinci
Data kelinci ditemukan pada indeks ke-2!
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\guided1.go"
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : kelinci
Masukkan data binatang indeks ke-3 : ayam
Masukkan data binatang indeks ke-4 : burung

Masukkan nama binatang yang mau dicari : singa
Data singa tidak ditemukan!

```

Penjelasan :

Program ini mengimplementasikan algoritma **Sequential Search** (pencarian berurutan) untuk mencari nama binatang dalam sebuah array bertipe string. Program mendefinisikan tipe alias `arrData` sebagai array dengan kapasitas 5 elemen string. Fungsi `seqSearch` menerima array dan string yang dicari, lalu menelusuri setiap elemen dari indeks 0 hingga 4 secara berurutan. Jika elemen

yang cocok ditemukan, fungsi langsung mengembalikan indeksinya dan berhenti menggunakan break, jika tidak ditemukan sama sekali maka fungsi mengembalikan nilai -1 sebagai penanda tidak ada.

Program meminta pengguna memasukkan 5 nama binatang satu per satu, lalu meminta nama binatang yang ingin dicari. Hasil pencarian kemudian dicetak dalam satu baris. Jika ditemukan, program mencetak pesan bahwa data ditemukan beserta indeksinya, jika tidak ditemukan program mencetak pesan bahwa data tidak ada.

2. [binarySearch.go]

```
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {

        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah
        }

        // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1

        // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }
}
```

```

        // jika data tidak ditemukan
        return -1
    }

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)

    // input isi array
    fmt.Println("Masukkan data terurut:")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    // input angka yang dicari
    fmt.Print("Angka yang dicari: ")
    fmt.Scan(&cari)

    // memanggil fungsi binary search
    hasil := binarySearch(data, cari)

    // output hasil pencarian
    if hasil != -1 {
        fmt.Println("Data ditemukan di index:", hasil)
    } else {
        fmt.Println("Data tidak ditemukan")
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\guided2.go"
Jumlah data: 5
Masukkan data terurut:
2, 5, 8, 12, 16
Angka yang dicari: 8
Data ditemukan di index: 2
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini mengimplementasikan algoritma **Binary Search** untuk mencari sebuah bilangan integer dalam kumpulan data yang sudah terurut. Berbeda dengan Sequential Search, Binary Search bekerja dengan cara membagi rentang pencarian menjadi dua bagian di setiap iterasi menggunakan tiga penanda yaitu kiri, kanan, dan tengah. Jika nilai tengah sama dengan yang dicari maka langsung dikembalikan indeksnya, jika nilai yang dicari lebih besar maka pencarian dilanjutkan ke bagian kanan, dan jika lebih kecil maka ke bagian kiri. Program menggunakan slice dinamis (`make([]int, n)`) sehingga ukuran array fleksibel sesuai input pengguna, dan fungsi `binarySearch` mengembalikan indeks posisi data jika ditemukan atau -1 jika tidak ada.

Program meminta pengguna memasukkan jumlah data terlebih dahulu, lalu mengisi data secara terurut membesar, kemudian memasukkan angka yang ingin dicari. Hasil pencarian dicetak dalam satu baris berupa indeks posisi jika ditemukan atau pesan tidak ditemukan.

3. [searchingArraystruct.go]

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        }
    }
    return found
}
```

```

        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
}

```

```

        if idxHasilCariSeqSearch > -1 {
            fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
            fmt.Println("Berikut Detail Datanya : ")
            fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
            fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
            fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
        } else if idxHasilCariSeqSearch == -1 {
            fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
        }
        fmt.Println()

        var idxHasilCariBinSearch int
        idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)

        fmt.Println("== HASIL BINARY SEARCH ==")
        if idxHasilCariBinSearch > -1 {
            fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
            fmt.Println("Berikut Detail Datanya : ")
            fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
            fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
            fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
        } else if idxHasilCariBinSearch == -1 {
            fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
        }
    }
}

```

Output :

```
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\guided3.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : ais
Masukkan NIM : 109082500162
Masukkan IPK : 3.93
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : melany
Masukkan NIM : 109087652017
Masukkan IPK : 3.5
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : cece
Masukkan NIM : 190875241729
Masukkan IPK : 3.1
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : hanifah
Masukkan NIM : 109082599718
Masukkan IPK : 2.7
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : nur
Masukkan NIM : 109028736152
Masukkan IPK : 3.7
-----

Masukkan NIM mahasiswa yang ingin dicari : 109082500162

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 109082500162 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : ais
NIM : 109082500162
IPK : 3.93

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 109082500162 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : ais
NIM : 109082500162
IPK : 3.93
PS D:\Praktikum\109082500162_Ais-Nurhanifah> |
```

Penjelasan :

Program ini mengimplementasikan dua algoritma pencarian sekaligus, yaitu **Sequential Search** dan **Binary Search**, untuk mencari data mahasiswa berdasarkan NIM dalam sebuah array struct. Program mendefinisikan struct mahasiswa yang memiliki tiga field yaitu nama (string), nim (int), dan IPK (float64), serta tipe alias arrMahasiswa sebagai array berkapasitas 5 elemen struct mahasiswa. Fungsi seqSearchStruct menelusuri array dari indeks 0 secara berurutan hingga NIM yang dicari ditemukan atau seluruh data habis diperiksa. Sementara fungsi binSearchStruct menggunakan tiga penanda kr (kiri), kn (kanan), dan med (tengah) untuk membagi rentang pencarian menjadi dua bagian di setiap iterasi, dengan syarat data NIM harus dimasukkan secara terurut membesar agar Binary Search bekerja dengan benar.

Program meminta pengguna memasukkan data 5 mahasiswa secara terurut berdasarkan NIM, lalu meminta NIM yang ingin dicari. Hasil pencarian ditampilkan dua kali, pertama dari Sequential Search dan kedua dari Binary

Search. Jika NIM ditemukan, kedua metode akan menampilkan indeks posisi beserta detail lengkap mahasiswa (nama, NIM, IPK). Jika tidak ditemukan, keduanya mencetak pesan data tidak ditemukan.

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT. Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 1: 1 2: 1 3: 2 7: 1 18: 1 19: 2

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    votes := make(map[int]int)
```

```

total := 0
valid := 0

for {
    var n int
    fmt.Scan(&n)
    if n == 0 {
        break
    }
    total++
    if n >= 1 && n <= 20 {
        valid++
        votes[n]++
    }
}

fmt.Printf("Suara masuk: %d\n", total)
fmt.Printf("Suara sah: %d\n", valid)

for i := 1; i <= 20; i++ {
    if votes[i] > 0 {
        fmt.Printf("%d: %d\n", i, votes[i])
    }
}
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\unguided1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program pilkart dirancang untuk membaca, memvalidasi, dan menghitung suara dalam pemilihan ketua RT yang memiliki 20 calon. Program membaca input berupa serangkaian bilangan bulat dalam satu baris secara berurutan menggunakan `fmt.Scan`, kemudian memeriksa setiap bilangan yang masuk. Setiap bilangan yang terbaca akan dihitung sebagai suara masuk, namun hanya bilangan dengan nilai antara 1 hingga 20 yang dianggap sebagai suara sah dan dicatat ke dalam `map[int]int` sebagai penampung perolehan suara tiap calon. Proses pembacaan berhenti otomatis ketika ditemukan bilangan bernilai 0 sebagai tanda akhir data.

Output program terdiri dari beberapa bagian yang terstruktur. Pertama, program mencetak total seluruh bilangan yang terbaca sebelum angka 0 sebagai "Suara masuk", lalu mencetak jumlah bilangan yang valid (bernilai 1–20) sebagai "Suara sah". Setelah itu, program melakukan iterasi dari calon nomor 1 hingga 20 secara berurutan, dan hanya mencetak calon yang memperoleh suara lebih dari nol beserta jumlah perolehan suaranya. Dengan pendekatan ini, pada contoh input 7 19 3 2 78 3 1 -3 18 19 0, terdapat 10 bilangan terbaca (suara masuk) namun hanya 8 yang valid karena 78 dan -3 berada di luar rentang 1–20, sehingga output menampilkan 6 calon yang mendapat suara yaitu calon 1, 2, 3, 7, 18, dan 19 dengan perolehan masing-masing sesuai frekuensi kemunculannya.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 Ketua RT: 3 Wakil ketua: 19

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    votes := make(map[int]int)
    total := 0
```

```

valid := 0

for {
    var n int
    fmt.Scan(&n)
    if n == 0 {
        break
    }
    total++
    if n >= 1 && n <= 20 {
        valid++
        votes[n]++
    }
}

fmt.Printf("Suara masuk: %d\n", total)
fmt.Printf("Suara sah: %d\n", valid)

// Cari ketua (suara terbanyak, nomor terkecil jika seri)
ketuaNo := -1
ketuaVote := -1
for i := 1; i <= 20; i++ {
    if votes[i] > ketuaVote {
        ketuaVote = votes[i]
        ketuaNo = i
    }
}

// Cari wakil (suara terbanyak kedua, bukan ketua, nomor terkecil jika seri)
wakilNo := -1
wakilVote := -1
for i := 1; i <= 20; i++ {
    if i == ketuaNo {
        continue
    }
    if votes[i] > wakilVote {
        wakilVote = votes[i]
        wakilNo = i
    }
}

fmt.Printf("Ketua RT: %d\n", ketuaNo)
fmt.Printf("Wakil ketua: %d\n", wakilNo)
}

```

Output :

```
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\unguided2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS D:\Praktikum\109082500162_Ais-Nurhanifah>
```

Penjelasan :

Program pilkart merupakan pengembangan dari program sebelumnya, dengan tambahan fitur untuk menentukan pemenang ketua RT dan wakil ketua RT. Sama seperti sebelumnya, program membaca input bilangan bulat satu per satu menggunakan `fmt.Scan`, memvalidasi setiap bilangan yang masuk, dan mencatatnya ke dalam `map[int]int`. Setelah semua suara terbaca, program melakukan dua kali iterasi dari nomor 1 hingga 20 untuk mencari pemenang. Iterasi pertama mencari calon dengan suara terbanyak sebagai ketua, dan iterasi kedua mencari calon dengan suara terbanyak berikutnya (di luar ketua) sebagai wakil. Penggunaan operator `>` (bukan `>=`) dalam perbandingan suara memastikan bahwa jika terjadi seri, calon dengan nomor peserta terkecil yang akan terpilih secara otomatis karena iterasi dilakukan dari nomor terkecil ke terbesar.

Output program terdiri dari empat baris yang terstruktur. Dua baris pertama sama seperti program sebelumnya, yaitu menampilkan total suara masuk dan jumlah suara sah. Dua baris berikutnya menampilkan hasil pencarian pemenang, yaitu "Ketua RT" dan "Wakil ketua" beserta nomor calonnya. Pada contoh input 7 19 3 2 78 3 1 -3 18 19 0, dari 8 suara sah yang tercatat, calon nomor 3 dan nomor 19 masing-masing memperoleh 2 suara sebagai perolehan tertinggi. Karena keduanya seri, calon nomor 3 terpilih sebagai Ketua RT karena memiliki nomor peserta lebih kecil, sedangkan calon nomor 19 menjadi Wakil Ketua sebagai calon dengan suara terbanyak kedua dari sisa calon yang ada.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan ini

```
package main
import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main(){
/* buatlah kode utama yang membaca baris pertama (n dan k). kemudian data diisi
oleh prosedur isiArray(n), dan pencarian oleh fungsi posisi(n,k), dan
setelah
itu output dicetak. */
}
```

```
func isiArray(n int){
/* I.S. terdefinisi integer n, dan sejumlah n data sudah siap pada piranti
masukan.
F.S. Array data berisi n (<=NMAX) bilangan */
}

func posisi(n, k int) int {
/* mengembalikan posisi k dalam array data dengan n elemen. Posisi dimulai
dari
posisi 0. Jika tidak ada kembalikan -1 */
}
```

Contoh masukan dan keluaran:

No	Masukan	Keluaran	Penjelasan
1	12 534 1 3 8 16 32 123 323 323 534 543 823 999	8	Data 534 berada pada posisi ke-8 dihitung dari awal data.
2	12 535 1 3 8 16 32 123 323 323 534 543 823 999	TIDAK ADA	

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000
```

```

var data [NMAX]int

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    low := 0
    high := n - 1
    for low <= high {
        mid := (low + high) / 2
        if data[mid] == k {
            return mid
        } else if data[mid] < k {
            low = mid + 1
        } else {
            high = mid - 1
        }
    }
    return -1
}

func main() {
    var n, k int
    fmt.Scan(&n, &k)
    isiArray(n)
    hasil := posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\unguided3.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 11 - modul 12\unguided3.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program pencarian posisi ini dirancang untuk menemukan keberadaan sebuah bilangan k dalam sekumpulan data integer positif yang sudah terurut membesar, menggunakan algoritma **Binary Search**. Program dibangun dengan tiga subprogram sesuai kerangka soal, yaitu fungsi main sebagai pengendali utama, prosedur isiArray(n) untuk membaca n buah data dari input ke dalam array global data, dan fungsi posisi(n, k) yang mengimplementasikan Binary Search untuk mencari posisi k . Algoritma Binary Search bekerja dengan membagi rentang pencarian menjadi dua bagian di setiap langkah, membandingkan nilai tengah (mid) dengan k , kemudian mempersempit rentang ke kiri jika k lebih kecil atau ke kanan jika k lebih besar. Pendekatan ini jauh lebih efisien dibanding pencarian linear karena kompleksitasnya hanya $O(\log n)$, sehingga mampu menangani data hingga 1.000.000 elemen dengan cepat.

Output program hanya terdiri dari satu baris saja. Jika bilangan k ditemukan dalam array, program mencetak posisi indeksinya yang dihitung mulai dari angka 0. Jika tidak ditemukan, fungsi posisi mengembalikan nilai -1 sebagai penanda, dan program mencetak "TIDAK ADA". Pada contoh pertama dengan input $k = 534$ dan data 1 3 8 16 32 123 323 323 534 543 823 999, Binary Search berhasil menemukan nilai 534 pada indeks ke-8 (dihitung dari 0) sehingga output yang tercetak adalah angka 8. Sebaliknya pada contoh kedua dengan $k = 535$, nilai tersebut tidak ada dalam array sehingga fungsi posisi mengembalikan -1 dan program mencetak "TIDAK ADA".

D. Kesimpulan

Dapat disimpulkan bahwa algoritma pencarian merupakan bagian penting dalam pemrograman untuk menemukan data tertentu dari sekumpulan data. Sequential Search adalah metode pencarian paling sederhana yang menelusuri data satu per satu secara linear tanpa syarat data terurut, namun kurang efisien untuk data berukuran besar karena kompleksitasnya $O(n)$. Binary Search adalah metode yang jauh lebih efisien dengan kompleksitas $O(\log n)$, namun memiliki syarat mutlak bahwa data harus sudah dalam keadaan terurut sebelum pencarian dilakukan.

Pemilihan algoritma pencarian yang tepat sangat bergantung pada kondisi dan ukuran data yang dihadapi. Jika data tidak terurut dan berukuran kecil maka Sequential Search sudah cukup memadai, namun jika data berukuran besar dan sudah terurut maka Binary Search adalah pilihan yang jauh lebih optimal. Penggunaan struct dalam program pencarian juga terbukti memudahkan pengelolaan data yang memiliki banyak atribut sekaligus, karena seluruh detail data dapat diakses langsung melalui satu indeks hasil pencarian. Dengan

memahami kedua algoritma ini beserta penerapannya dalam bahasa Go, pemrogram dapat memilih dan mengimplementasikan metode pencarian yang paling sesuai dengan kebutuhan programnya.